MODULE 1 ASSIGNMENT

"How the Internet Delivers a Web Page"

Student Name:	Tony Gilson
Roll No:	58
Class:	CSA
Course:	PCCST501 – Computer Networks
Module:	1
Website chosen:	google.com (Google Search)

1. Introduction

When a user types `www.google.com` into a browser's address bar and presses Enter, a rapid sequence of events unfolds across several layers of the Internet's protocol stack. Although the entire process typically completes in under a second, it involves name resolution, connection establishment, secure data exchange, and rendering — each handled by a different layer of the TCP/IP model working together. This report traces that journey step by step, using Google Search as the running example.

2. The TCP/IP Protocol Stack

The TCP/IP model organizes network communication into four layers. Each layer has a distinct job and hands off data to the layer below (on the sending side) or above (on the receiving side).

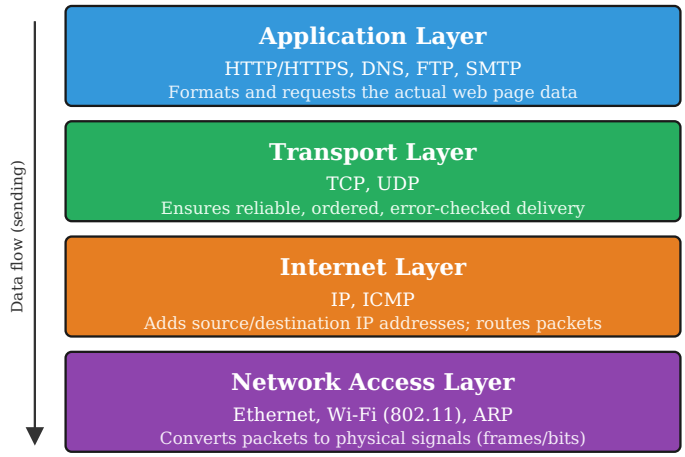

Figure 1: TCP/IP four-layer protocol stack

Role of each layer

- **Application Layer** – This is where the browser operates. It generates the HTTP/HTTPS request for the Google Search page and interprets the HTML/CSS/JavaScript that comes back. DNS also functions at this layer, translating `google.com` into an IP address before the request is sent.
- **Transport Layer** – TCP takes the HTTP request, breaks it into segments, numbers them, and guarantees they arrive at Google's server complete and in order. It also manages the three-way handshake that opens the connection.
- **Internet Layer** – IP wraps each TCP segment into a packet and attaches the source IP (the user's device) and destination IP (Google's server). Routers use this layer to forward the packet across the many networks between the user and Google's data center.
- **Network Access Layer** – Converts IP packets into frames suitable for the physical medium in use — Wi-Fi radio signals or Ethernet electrical/optical signals — and handles the actual transmission over the local network hop.

Data encapsulation across the layers

As the search request travels down the stack on the sending side, each layer wraps the data from the layer above it with its own header — a process called **encapsulation**. The Application layer hands over the raw HTTP request; the Transport layer wraps it in a TCP segment header (adding source/destination port numbers); the Internet layer wraps that in an IP packet header (adding source/destination IP addresses); and the Network Access layer wraps the whole thing in a frame header (adding MAC addresses) before converting it to electrical, radio, or optical signals. At Google's server, the exact reverse process — **de-encapsulation** — strips each header away, layer by layer, until the original HTTP request is recovered.

Why TCP and not UDP for this request

The Transport layer could use either TCP or UDP. Google Search deliberately uses TCP (via HTTPS) rather than UDP because a corrupted or missing search results page is unacceptable — the user needs the complete, correctly-ordered HTML. UDP, which offers no delivery guarantees, is reserved for use cases where speed matters more than perfection, such as live video calls or DNS lookups themselves.

3. Role of DNS

Browsers and routers do not understand domain names — they route traffic using numeric IP addresses. The Domain Name System (DNS) is the Internet's directory service that performs this translation.

1. The browser first checks its own cache, then the operating system's cache, for a recent record of `google.com`.
2. If not cached, a DNS resolver (usually run by the ISP or a public service such as 8.8.8.8) is queried.
3. The resolver walks the DNS hierarchy: a **root server** points it to the **.com TLD server**, which points it to Google's **authoritative name server**.
4. The authoritative server returns the actual IP address, for example `142.250.193.174`.
5. This IP is cached locally (subject to a TTL) and handed to the browser so the connection can proceed.

Without DNS, users would need to memorize numeric addresses for every website they visit, which is why DNS is often called the "phonebook of the Internet."

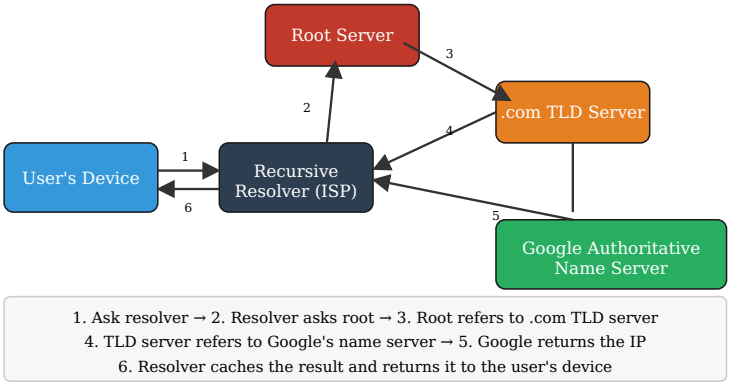

Figure 3: Hierarchical DNS resolution process for google.com

Types of DNS records involved

Record Type	Purpose
A	Maps a domain name to an IPv4 address (e.g. google.com → 142.250.193.174)
AAAA	Maps a domain name to an IPv6 address
CNAME	Aliases one domain name to another, canonical domain name
NS	Specifies which name servers are authoritative for the domain
TTL	Not a record type itself, but a value telling resolvers how long to cache a record

4. HTTP: Requesting and Receiving the Webpage

Once the browser has Google's IP address and a TCP connection is open, it uses HTTP (or HTTPS, the encrypted version) to actually fetch the page:

1. The browser sends a **GET** request, e.g. `GET /search?q=example HTTP/1.1`, along with headers such as `Host: www.google.com` and `User-Agent`.
2. Google's web server processes the request and returns an **HTTP response**: a status line (e.g. `200 OK`), response headers, and the body — the HTML document itself.
3. The browser parses the HTML and issues further GET requests for embedded resources: CSS stylesheets, JavaScript files, images, and fonts.
4. Once enough resources have arrived, the browser renders the page on screen.

Sample HTTP request and response

A simplified version of what the browser actually sends and receives looks like this:

Request (browser → server)

```
GET /search?q=example HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0
Accept: text/html
Connection: keep-alive
```

Response (server → browser)

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 54823
Set-Cookie: session_id=abc123; Secure; HttpOnly
<!DOCTYPE html>…(page content)…
```

HTTP vs HTTPS

In practice, a browser visiting Google does not use plain HTTP at all — it uses **HTTPS**, which wraps the same HTTP request/response cycle inside a TLS (Transport Layer Security) encryption layer. Before any HTTP data is exchanged, the browser and server perform a TLS handshake

to agree on encryption keys and verify the server's identity certificate. This prevents attackers on the network from reading or tampering with the search query or the results, which matters since search terms can reveal sensitive personal information.

Common HTTP status codes

The status line in the response tells the browser how the request was handled:

Code	Meaning	Example on google.com
200 OK	Request succeeded, content returned	Search results page loads normally
301 / 302	Redirect to another URL	`google.com` redirecting to `www.google.com`
304	Not modified – use cached version	Re-loading a cached logo image
404	Requested resource not found	A broken or mistyped search link
500	Internal server error	A temporary fault on Google's servers

5. How TCP Provides Reliable Delivery

HTTP relies entirely on TCP beneath it to make sure the request and response actually arrive correctly. TCP achieves reliability through several mechanisms:

- **Three-way handshake** – SYN, SYN-ACK, ACK segments establish a connection before any data is exchanged, so both sides agree they are ready.
- **Sequence numbers** – Every byte sent is numbered, allowing the receiver to reassemble segments in the correct order even if they arrive out of sequence.
- **Acknowledgements (ACKs)** – The receiver confirms receipt of data; missing ACKs signal that a segment was lost.
- **Retransmission** – If an ACK is not received within a timeout, TCP resends the lost segment.
- **Flow control** – A sliding window mechanism prevents the sender from overwhelming a slower receiver.
- **Checksums** – Detect corrupted data so it can be discarded and retransmitted.

6. Client–Server or Peer-to-Peer?

Google Search follows a **Client–Server architecture**. The user's browser (client) always initiates the request, and Google operates centralized, dedicated data-center servers that respond to requests, index the web, and serve results. Clients never communicate directly with one another to fetch search results — every request is routed through Google's infrastructure. This is the defining trait of Client–Server systems, as opposed to Peer-to-Peer (P2P) systems such as BitTorrent, where every participating node can act as both a client and a server and content is exchanged directly between peers without a central authority.

Aspect	Client–Server (Google Search)	Peer-to-Peer (e.g. BitTorrent)
Roles	Fixed – client requests, server responds	Interchangeable – every node is both client and server
Point of control	Centralized at Google's data centers	Decentralized across participating peers
Scalability approach	Achieved by adding more/larger servers	Achieved automatically as more peers join
Single point of failure	Yes, if the server/data center goes down	No, network survives individual peer loss
Typical use	Search engines, banking, email	File sharing, some blockchain networks

7. HTTP vs FTP

Aspect	HTTP	FTP
Purpose	Transfers web pages and resources for browsing/display	Transfers files between a client and server for storage/download
Connections used	Single connection carries both requests and data	Two separate connections: control (port 21) and data (port 20)
State	Stateless by default (state added via cookies)	Stateful – session persists for the duration of the connection
Authentication	Often none required for public content	Typically requires a username and password
Typical use case	Loading a webpage such as google.com	Uploading a website's files to a hosting server

It is worth noting that FTP's separate control and data connections can operate in two modes: **active mode**, where the server opens a new connection back to the client to send data, and **passive mode**, where the client initiates both connections — the latter is more firewall-friendly and is the default in most modern FTP clients. HTTP has no such distinction since it relies on a single, simpler connection model, which is one reason it became the protocol of choice for everyday browsing while FTP remains common mainly for bulk file transfer and server administration tasks.

8. How Cookies Improve User Experience

HTTP is stateless, meaning the server does not naturally remember a user between requests. Cookies solve this by letting the server store a small piece of data on the user's browser, which is sent back with every subsequent request to that domain.

Real-world example: When a user signs in to their Google account and searches, Google sets a session cookie. This cookie lets the user stay logged in while navigating from Search to Gmail to YouTube without re-entering credentials on every page, and it also allows Google to remember preferences such as language and safe-search settings across visits.

9. End-to-End Communication Flow

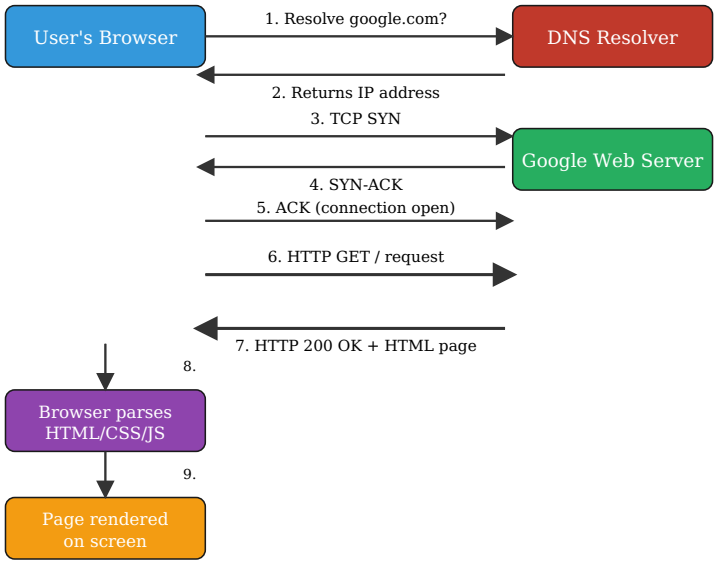

Figure 2: Complete browser-to-server communication flow for loading google.com

10. Conclusion

Loading a single webpage like google.com depends on a tightly coordinated stack of application-layer protocols working invisibly in the background. DNS turns a memorable name into a routable address, TCP guarantees that data survives the journey intact, IP finds the path across networks, and HTTP structures the actual conversation between browser and server. Cookies then layer statefulness on top of an inherently stateless protocol, enabling personalization and persistent logins. Understanding these application-layer protocols matters because they are the direct interface between users and the Internet's infrastructure: every search, login, and page load depends on them functioning correctly, quickly, and securely. Without this layered design, the modern web — built on billions of daily HTTP requests — simply could not scale or remain reliable.

11. References

- Kurose, J. F., & Ross, K. W. – *Computer Networking: A Top-Down Approach*
- Forouzan, B. A. – *Data Communications and Networking*
- Mozilla Developer Network (MDN Web Docs) – HTTP documentation
- IETF RFC 793 – Transmission Control Protocol
- IETF RFC 1035 – Domain Names: Implementation and Specification